

Government Polytechnic, Pune -16

(An Autonomous Institute of Government of Maharashtra)

Department of Computer Engineering

(Academic Year : 2026-2027)



**A Micro-Project
Report On**

"CLIENT-SERVER CHAT APPLICATION"

Submitted by :

Group Members	Enrollment No.
Avinash Jambhale	2406085
Siddharth Lokhande	2406119
Dnyaneshwar Mengal	2306117

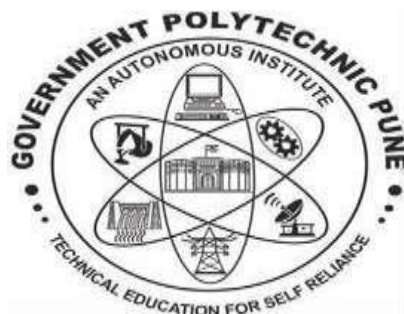
Under the Guidance of

Smt S.B.Gosavi

Government Polytechnic Pune -16
(An Autonomous Institute of Government of Maharashtra)

University Road, Shivajinagar, Pune-411016

Department of Computer Engineering Academic Year 2026-27



CERTIFICATE

This is to certify that the micro-project work entitled “CLIENT-SERVER CHAT APPLICATION” is a bonafide work carried out by

Group Members	Enrollment No.
Avinash Jambhale	2406085
Siddharth Lokhande	2406119
Dnyaneshwar Mengal	2306117

of class Third Year in partial fulfilment of the requirement for the completion of Course of **Java Programming** (CM41205) in Diploma in Computer Engineering from Government Polytechnic Pune.

Course Guide
(Smt.S.B.Gosavi)

Head Of Department
(Smt. J.R.Hange)

Principal
(Dr. R.K.Patil)

ACKNOWLEDGMENT

The success of any project depends largely on the encouragement and guidelines of many others. This research project would not have been possible without their support. We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project.

First and foremost, we wish to record our sincere gratitude to the Management of this college and to our Respected Principal **Dr. R.K.Patil**, for his constant support and encouragement in the preparation of this report and for the availability of library facilities needed to prepare this report.

Our sincere thanks to **Smt. J.R.Hange**, Head of Computer Department, Government Polytechnic, Pune who was abundantly helpful and offered invaluable assistance, support and guidance.

We express our sincere gratitude to our Guide, **Smt S.B.Gosavi**, for guiding us in investigations of this project and in carrying out experimental work. Our numerous discussions were extremely helpful. We are highly indebted to her for her guidance and constant supervision as well as for providing necessary information regarding the project & also for his support in completing the project. We hold him in esteem for guidance, encouragement and inspiration received from him.

We wish to thank our parents for financing our studies and helping us throughout our life for achieving perfection and excellence. Their personal help in making this report and project worth presentation is gratefully acknowledged. Last but not the least we thank the Almighty for continuous strength we were bestowed for completion of this project.

Sr. No.	Title of Content
1.	Abstract
2.	Introduction
3.	Objective
4.	System Architecture
5.	Implementation Details
6.	Technologies Used
7.	Project Output
8.	Advantages and Disadvantages
9.	Challenges Faced
10.	Conclusion
11.	Reference

ABSTRACT

This micro-project involves the design and implementation of a multithreaded Client-Server Chat Application using core Java technologies. The application provides a platform for multiple users to communicate in real-time within a shared chat room. The project leverages Java Sockets for network communication and Java Swing for the graphical user interface (GUI).

The server is designed to handle multiple client connections concurrently, using a dedicated thread for each connected client. This ensures that the server remains responsive and can manage simultaneous conversations without blocking. The server maintains a log of key events, such as client connections and disconnections.

The client application features a user-friendly interface where users can enter their name, connect to the server, send messages, and view the ongoing conversation. A separate listener thread on the client side ensures that the UI remains responsive while continuously receiving messages from the server.

This project provides hands-on experience with fundamental networking concepts, multithreading, and GUI programming in Java. It serves as a practical demonstration of building a distributed application and managing concurrent operations in a networked environment.

Key Features:

- Real-time messaging between multiple clients.
- Multithreaded server to handle concurrent client connections.
- GUI-based client application built with Java Swing.
- Centralized server for message broadcasting.
- Server log for monitoring client activity.

INTRODUCTION

In today's interconnected world, communication applications are essential. Client-server architecture is a fundamental model for building such networked applications, where a central server provides services to multiple clients. This project focuses on creating a classic example of this architecture: a text-based chat application.

Using Java's powerful networking and concurrency APIs, this project demonstrates how to build a robust system where multiple users can join a chat room and exchange messages in real-time. The server application acts as a central hub, accepting connections from clients and broadcasting messages to all participants. The client application provides the user interface for interaction.

This project is an excellent exercise in applying core Java concepts like Object-Oriented Programming, I/O Streams, Socket Programming, Multithreading, and GUI development with Swing. It provides a foundational understanding of how distributed systems operate and the challenges involved in managing concurrent user interactions.

OBJECTIVES

- To design and develop a functional multi-client chat application using Java.
- To implement a server that can accept and manage multiple client connections simultaneously using multithreading.
- To create a user-friendly client-side graphical user interface (GUI) using Java Swing.
- To establish reliable two-way communication between the client and server using Java Sockets and I/O Streams.
- To implement a message broadcasting mechanism on the server to send a message from one client to all other connected clients.
- To ensure the client GUI remains responsive by handling network operations in a separate thread.
- To learn and apply concepts of exception handling and thread synchronization in a practical networking application.

SYSTEM ARCHITECTURE

The application follows a classic client-server model. The architecture is composed of two primary components: a single **Server** and multiple **Clients**.

1. Server:

- **ServerSocket:** The server starts by creating a `ServerSocket` that listens for incoming connection requests on a specific port (e.g., 9999).
- **Main Thread:** The server's main thread runs an infinite loop, waiting for new clients to connect using the `serverSocket.accept()` method.
- **ClientHandler Thread:** When a new client connects, the server creates a new `ClientHandler` object and spawns a dedicated thread for it. This thread is responsible for handling all communication with that specific client, allowing the main thread to immediately go back to listening for more connections. This multithreaded design is crucial for serving multiple clients concurrently.
- **Client List:** The server maintains a synchronized list of all active `ClientHandler` instances.
- **Broadcasting:** When a `ClientHandler` thread receives a message from its client, it passes the message to a synchronized `broadcastMessage` method on the server, which then relays the message to all other connected clients.

2. Client:

- **Socket:** The client initiates a connection to the server's IP address and port by creating a `Socket` object.
- **GUI (Event Dispatch Thread):** The user interface is built with Java Swing and runs on the Event Dispatch Thread (EDT). It consists of a text area to display messages, a text field for input, and a send button.
- **Main Thread:** The main client thread handles GUI setup and initiates the network connection.
- **ServerListener Thread:** After connecting, the client starts a new background thread (`ServerListener`) dedicated to listening for incoming messages from the server via its `socket.getInputStream()`. This prevents the GUI from freezing while waiting for data. When a message is received, it uses `SwingUtilities.invokeLater()` to safely update the message area on the EDT.
- **Sending Messages:** When the user sends a message, an action listener on the EDT writes the message to the `socket.getOutputStream()`.

IMPLEMENTATION DETAILS

1. Server-Side Connection Handling

The server uses a `ServerSocket` to listen for clients. For each new connection, it creates and starts a `ClientHandler` thread.

```
// Snippet from Server.java public Server() {
    // ... GUI Setup ... try {
        ServerSocket serverSocket = new ServerSocket(9999); log("Waiting for clients on
        port 9999...");

        while (true) {
            Socket clientSocket = serverSocket.accept(); log("Client connected: " +
            clientSocket.getInetAddress().getHostAddress());

            ClientHandler clientHandler = new ClientHandler(clientSocket); clients.add(clientHandler);
            new Thread(clientHandler).start();
        }
    } catch (IOException e) {
        log("Error starting server: " + e.getMessage());
    }
}
```

2. Message Broadcasting (Server)

The broadcastMessage method iterates through the list of connected clients and sends the message to everyone except the original sender. The method is synchronized to prevent concurrent modification issues.

```
// Snippet from Server.java
private synchronized void broadcastMessage(String message, ClientHandler sender) {
    log("Broadcasting: " + message);
    for (ClientHandler client : clients) {
        // Send to all clients except the sender if (client != sender)
        {
            client.sendMessage(message);
        }
    }
}
```

3. Client-Side Connection and I/O Setup

The client connects to the server using a Socket and sets up PrintWriter for sending data and BufferedReader for receiving data.

```
// Snippet from Client.java public Client() {
    // ... GUI Setup and input dialogs ... try {
        socket = new Socket(serverIp, 9999);
        out = new PrintWriter(socket.getOutputStream(), true); in = new
        BufferedReader(new
        InputStreamReader(socket.getInputStream()));
```

```

        // Send client name to server as the first message out.println(clientName);

        // Start a new thread to listen for messages from the server new Thread(new
        ServerListener()).start();

    } catch (IOException e) {
        JOptionPane.showMessageDialog(frame, "Could not connect...", "Connection Error", ...);
        System.exit(1);
    }
}

```

4. Asynchronous Message Receiving (Client)

The ServerListener inner class runs on a separate thread, continuously reading messages from the server and updating the GUI safely.

```

// Snippet from Client.java
class ServerListener implements Runnable { @Override
    public void run() { try {
        String serverMessage;
        while ((serverMessage = in.readLine()) != null) { final String msg =
            serverMessage;
            // Use invokeLater to update GUI from a non-EDT thread SwingUtilities.invokeLater(() ->
            messageArea.append(msg +
"\n"));
        }
    } catch (IOException e) {
        SwingUtilities.invokeLater(() -> messageArea.append("Connection
to server lost.\n"));
    }
}
}

```

TECHNOLOGIES USED

Technology	Purpose
Java (JDK 8+)	Core programming language for the application.
Java Swing	For building the graphical user interface (GUI).
Java Sockets API	For network communication between client and server.
Java AWT	For event handling and basic graphics.
Java Threading	For concurrent execution on the server and responsive UI on the client.

Development Tools

Tool	Purpose
Eclipse/IntelliJ IDEA	Integrated Development Environment.
Git	Version control system.

Key Java Packages Utilized

- `javax.swing.*`: For all GUI components (JFrame, JTextArea, etc.).
- `java.awt.*`: For layout managers (BorderLayout) and fonts.
- `java.awt.event.*`: For handling user actions like button clicks.
- `java.net.*`: For Socket and ServerSocket classes.
- `java.io.*`: For input/output streams (PrintWriter, BufferedReader).

PROGRAM

Client :

```
import javax.swing.*;
```

```
import javax.swing.border.EmptyBorder;

import javax.swing.text.*;

import java.awt.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.awt.event.WindowAdapter;

import java.awt.event.WindowEvent;

import java.io.BufferedReader;

import java.io.IOException;

import java.io.InputStreamReader;

import java.io.PrintWriter;

import java.net.Socket;
```

```
public class Client {

    private JFrame frame;

    private JTextPane messageArea;

    private JTextField messageField;

    private JButton sendButton;

    private PrintWriter out;

    private BufferedReader in;

    private Socket socket;

    private String clientName;

    private StyledDocument messageDoc;
```

```
// Modern UI Colors

private static final Color BACKGROUND_COLOR = new Color(33, 37, 41);

private static final Color CARD_COLOR = new Color(52, 58, 64);

private static final Color TEXT_COLOR = new Color(248, 249, 250);

private static final Color ACCENT_COLOR = new Color(13, 110, 253);

private static final Color ADMIN_COLOR = new Color(220, 53, 69);

private static final Color SUCCESS_COLOR = new Color(25, 135, 84);

private static final Font MODERN_FONT = new Font("Segoe UI", Font.PLAIN, 14);

private static final Font TITLE_FONT = new Font("Segoe UI", Font.BOLD, 18);

public Client() {

    try {

        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeel());

    } catch (Exception e)

        { e.printStackTrace(

            );

        }

    */

    // Prompt for Server IP and Client Name with modern dialogs

    String serverIp = showModernInputDialog("Enter Server IP Address:", "127.0.0.1");

    if (serverIp == null || serverIp.trim().isEmpty()) {

        System.exit(0);

    }
```

```
clientName = showModernInputDialog("Enter your name:", "Guest");

if (clientName == null || clientName.trim().isEmpty()) {
    clientName = "Guest" + (int)(Math.random() * 1000);
}

// --- Modern GUI Setup ---

frame = new JFrame(" Chat Client - " + clientName);

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

frame.setSize(600, 500);

frame.setLayout(new BorderLayout());

frame.setLocationRelativeTo(null);

frame.getContentPane().setBackground(BACKGROUND_COLOR);

// Create main panel with modern styling

JPanel mainPanel = new JPanel(new BorderLayout());

mainPanel.setBackground(BACKGROUND_COLOR);

mainPanel.setBorder(new EmptyBorder(10, 10, 10,

10));

// Create header panel

JPanel headerPanel = createHeaderPanel();

mainPanel.add(headerPanel, BorderLayout.NORTH

// Setup message area with modern styling

messageArea = new JTextPane();
```

```
messageArea.setEditable(false);

messageArea.setFont(MODERN_FONT);

messageArea.setBackground(CARD_COLOR);

messageArea.setForeground(TEXT_COLOR);

messageArea.setBorder(new EmptyBorder(10, 10, 10,
10));

messageDoc = messageArea.getStyledDocument();

JScrollPane scrollPane = new JScrollPane(messageArea);
scrollPane.setBorder(BorderFactory.createLineBorder(new Color(108, 117, 125),
1)); scrollPane.getViewPort().setBackground(CARD_COLOR);

scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_AS_NEED
ED);

scrollPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEV
ER);

mainPanel.add(scrollPane, BorderLayout.CENTER);

// Create bottom panel with modern input

JPanel bottomPanel = createBottomPanel();

mainPanel.add(bottomPanel, BorderLayout.SOUTH);

frame.add(mainPanel

// --- Action Listeners ---

sendButton.addActionListener(e -> sendMessage());
```

```
messageField.addActionListener(e -> sendMessage());
```

```
frame.addWindowListener(new WindowAdapter()
```

```
{ @Override
```

```
public void windowClosing(WindowEvent e)
```

```
{ if (out != null) {
```

```
    out.println(clientName + " is disconnecting.");
```

```
}
```

```
try {
```

```
    if (socket != null) {
```

```
        socket.close();
```

```
    }
```

```
} catch (IOException ioException) {
```

```
    // Handle error
```

```
}
```

```
}
```

```
});
```

```
frame.setVisible(true
```

```
// --- Networking
```

```
---try {
```

```
    socket = new Socket(serverIp, 9999);
```

```
    out = new PrintWriter(socket.getOutputStream(), true);
```

```

        in = new BufferedReader(new InputStreamReader(socket.getInputStream()

// Send client name to server as the first message

        out.println(clientName

// Start a new thread to listen for messages from the server

        new Thread(new ServerListener()).start();

// Add welcome message

        appendStyledMessage("菟'芙'萝' Welcome to the chat, " + clientName + "! *菟`萝'芙",
SUCCESS_COLOR, true);

    } catch (IOException e) {

        JOptionPane.showMessageDialog(frame, "Could not connect to the server: " +
e.getMessage(), "Connection Error", JOptionPane.ERROR_MESSAGE);

        System.exit(1);

    }

private String showModernInputDialog(String message, String defaultValue)

    { JPanel panel = new JPanel(new BorderLayout(10, 10));

    panel.setBackground(CARD_COLOR);

    panel.setBorder(new EmptyBorder(20, 20, 20, 20));

    JLabel label = new JLabel(message);

    label.setForeground(TEXT_COLOR);

    label.setFont(MODERN_FONT

    JTextField textField = new JTextField(defaultValue, 20);

    textField.setFont(MODERN_FONT);

    textField.setBackground(BACKGROUND_COLOR);

```

```
textField.setForeground(TEXT_COLOR);

textField.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(new Color(108, 117, 125), 1),
    new EmptyBorder(8, 8, 8, 8)));

panel.add(label, BorderLayout.NORTH);

panel.add(textField, BorderLayout.CENTER);


int result = JOptionPane.showConfirmDialog(null, panel, "Chat Setup",
    JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);

if (result == JOptionPane.OK_OPTION)
    { return textField.getText();
    }

return null;
}


private JPanel createHeaderPanel() {

    JPanel headerPanel = new JPanel(new BorderLayout());

    headerPanel.setBackground(BACKGROUND_COLOR);

    headerPanel.setBorder(new EmptyBorder(0, 0, 15, 0));

    JLabel titleLabel = new JLabel(" Chat Room", SwingConstants.CENTER);

    titleLabel.setFont(TITLE_FONT);

    titleLabel.setForeground(TEXT_COLOR);
```

```
JLabel statusLabel = new JLabel(" Connected as " + clientName,  
SwingConstants.CENTER);
```

```
statusLabel.setFont(new Font("Segoe UI", Font.ITALIC,  
12)); statusLabel.setForeground(SUCCESS_COLOR);
```

```
headerPanel.add(titleLabel, BorderLayout.NORTH);
```

```
headerPanel.add(statusLabel, BorderLayout.CENTER);
```

```
return headerPanel;
```

```
}
```

```
private JPanel createBottomPanel() {
```

```
JPanel bottomPanel = new JPanel(new BorderLayout(10, 0));
```

```
bottomPanel.setBackground(BACKGROUND_COLOR);
```

```
bottomPanel.setBorder(new EmptyBorder(15, 0, 0, 0));
```

```
messageField = new JTextField();
```

```
messageField.setFont(MODERN_FONT);
```

```
messageField.setBackground(CARD_COLOR);
```

```
messageField.setForeground(TEXT_COLOR);
```

```
messageField.setBorder(BorderFactory.createCompoundBorder(  

```

```
BorderFactory.createLineBorder(new Color(108, 117, 125), 1),
```

```
new EmptyBorder(10, 10, 10, 10)));
```

```
messageField.setCaretColor(TEXT_COLOR);
```

```
sendButton = new JButton("   -  Send");
```

```
sendButton.setFont(new Font("Segoe UI", Font.BOLD,
14)); sendButton.setBackground(ACCENT_COLOR);
sendButton.setForeground(Color.WHITE);
sendButton.setBorder(new EmptyBorder(10, 20, 10, 20));
sendButton.setFocusPainted(false);
sendButton.setCursor(new Cursor(Cursor.HAND_CURSOR));

// Add hover effect
sendButton.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mouseEntered(java.awt.event.MouseEvent evt) {
        sendButton.setBackground(ACCENT_COLOR.brighter());
    }
    public void mouseExited(java.awt.event.MouseEvent evt)
    { sendButton.setBackground(ACCENT_COLOR);
    }
});

bottomPanel.add(messageField, BorderLayout.CENTER);
bottomPanel.add(sendButton, BorderLayout.EAST);

return bottomPanel;
}
```

```

// --- Inner class to listen for server messages

---class ServerListener implements Runnable {

    @Override

    public void run() {

        try {

            String serverMessage;

            while ((serverMessage = in.readLine()) != null) {

                final String msg = serverMessage;

                SwingUtilities.invokeLater() -> {

                    // Check if it's an admin message

                    if (msg.startsWith("[ADMIN]")) {

                        String adminMsg = msg.substring(7); // Remove [ADMIN] prefix

                        appendStyledMessage("🟡✗ ADMIN: " + adminMsg, ADMIN_COLOR, true);

                    } else {

                        // Regular message

                        appendStyledMessage(msg, TEXT_COLOR, false);

                    }

                });

            }

        } catch (IOException e) {

            SwingUtilities.invokeLater() ->

                appendStyledMessage("+ Connection to server lost.", ADMIN_COLOR, true));

        }

    }

}

```

```
        }  
    }  
}  
  
public static void main(String[] args)  
{  
    new Client();  
}
```

Server :

```
import javax.swing.*;  
  
import javax.swing.border.EmptyBorder;  
  
import java.awt.*;  
  
import java.awt.event.ActionEvent;  
  
import java.awt.event.ActionListener;  
  
import java.io.*;  
  
import java.net.ServerSocket;  
  
import java.net.Socket;  
  
import java.util.ArrayList;  
  
import java.util.List;  
  
  
public class Server {  
  
    private JFrame frame;  
  
    private JTextArea messageArea;
```

```
private JTextField adminMessageField;

private JButton sendAdminButton;

private List<ClientHandler> clients = new ArrayList<>();


// Modern Admin UI Colors (different from client)

private static final Color ADMIN_BACKGROUND = new Color(25, 25, 28);

private static final Color ADMIN_CARD = new Color(40, 40, 43);

private static final Color ADMIN_TEXT = new Color(255, 255, 255);

private static final Color ADMIN_ACCENT = new Color(255, 87, 51);

private static final Color ADMIN_SUCCESS = new Color(76, 175, 80);

private static final Color ADMIN_WARNING = new Color(255, 193,

7);

private static final Font ADMIN_FONT = new Font("JetBrains Mono", Font.PLAIN, 13);

private static final Font ADMIN_TITLE_FONT = new Font("JetBrains Mono", Font.BOLD,
16);


public Server() {

    // --- Modern Admin GUI Setup ---

    frame = new JFrame(" Chat Server - Admin Console");

    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    frame.setSize(700, 600);

    frame.setLayout(new BorderLayout());

    frame.setLocationRelativeTo(null);

    frame.getContentPane().setBackground(ADMIN_BACKGROUND);
```

```
// Create main panel

JPanel mainPanel = new JPanel(new BorderLayout());

mainPanel.setBackground(ADMIN_BACKGROUND);

mainPanel.setBorder(new EmptyBorder(15, 15, 15,
15));


// Create header panel

JPanel headerPanel = createAdminHeaderPanel();

mainPanel.add(headerPanel, BorderLayout.NORTH);


// Setup server log area

messageArea = new JTextArea();

messageArea.setEditable(false);

messageArea.setFont(ADMIN_FONT);

messageArea.setBackground(ADMIN_CARD);

messageArea.setForeground(ADMIN_TEXT);

messageArea.setBorder(new EmptyBorder(10, 10, 10,
10));


JScrollPane scrollPane = new
JScrollPane(messageArea);

scrollPane.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(ADMIN_ACCENT, 2),
    " Server Log",
    0, 0, ADMIN_FONT, ADMIN_TEXT));

scrollPane.getViewport().setBackground(ADMIN_CARD);
```

```
mainPanel.add(scrollPane, BorderLayout.CENTER);

// Create admin control panel

JPanel controlPanel = createAdminControlPanel();

mainPanel.add(controlPanel, BorderLayout.SOUTH

frame.add(mainPanel);

frame.setVisible(true);


log(" Server started.");


// --- Server Logic

---try {

    ServerSocket serverSocket = new

    ServerSocket(9999); log(" Waiting for clients on port

    while (true) {

        Socket clientSocket = serverSocket.accept();

        log(" Client connected: " + clientSocket.getInetAddress().getHostAddress());


        ClientHandler clientHandler = new

        ClientHandler(clientSocket); clients.add(clientHandler);

        new Thread(clientHandler).start();

    }

} catch (IOException e) {

    log("Error starting server: " + e.getMessage());
```

```
        e.printStackTrace();
    }

    private JPanel createAdminHeaderPanel() {

        JPanel headerPanel = new JPanel(new BorderLayout());

        headerPanel.setBackground(ADMIN_BACKGROUND);

        headerPanel.setBorder(new EmptyBorder(0, 0, 20, 0));


        JLabel titleLabel = new JLabel(" ADMIN CONTROL CENTER",
            SwingConstants.CENTER);

        titleLabel.setFont(ADMIN_TITLE_FONT);

        titleLabel.setForeground(ADMIN_ACCENT);


        JLabel statusLabel = new JLabel("Server Online - Managing Chat Room",
            SwingConstants.CENTER);

        statusLabel.setFont(new Font("JetBrains Mono", Font.ITALIC,
            12)); statusLabel.setForeground(ADMIN_SUCCESS);


        headerPanel.add(titleLabel, BorderLayout.NORTH);

        headerPanel.add(statusLabel, BorderLayout.CENTER);


        return headerPanel;

    }

    private JPanel createAdminControlPanel() {

        JPanel controlPanel = new JPanel(new BorderLayout());
```

```
controlPanel.setBackground(ADMIN_BACKGROUND);

controlPanel.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(ADMIN_WARNING, 2),
    '🔔 Admin Broadcast',
    0, 0, ADMIN_FONT, ADMIN_TEXT));

controlPanel.setBorder(BorderFactory.createCompoundBorder( controlPanel.g
    etBorder(),
    new EmptyBorder(10, 10, 10, 10)));

JPanel inputPanel = new JPanel(new BorderLayout(10, 0));
inputPanel.setBackground(ADMIN_BACKGROUND);

adminMessageField = new JTextField();
adminMessageField.setFont(ADMIN_FONT);
adminMessageField.setBackground(ADMIN_CARD);
adminMessageField.setForeground(ADMIN_TEXT);
adminMessageField.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(new Color(128, 128, 128), 1),
    new EmptyBorder(8, 8, 8, 8)));
adminMessageField.setCaretColor(ADMIN_TEXT);

sendAdminButton = new JButton(" BROADCAST");
sendAdminButton.setFont(new Font("JetBrains Mono", Font.BOLD, 12));
sendAdminButton.setBackground(ADMIN_ACCENT);
```

```

String message = adminMessageField.getText().trim();

if (!message.isEmpty()) {

    String adminMsg = "[ADMIN] " + message;

    log(" Broadcasting admin message: " +
        message);

    // Broadcast to all clients with [ADMIN] prefix
    synchronized(clients) {

        for (ClientHandler client : clients) {

            client.sendMessage(adminMsg);

        }

    }

    adminMessageField.setText("");

}

private void log(String message) {

    // Append message to the server's text area with timestamp

    String timestamp = java.time.LocalTime.now().toString().substring(0,
        8); SwingUtilities.invokeLater(() ->

        messageArea.append("[ " + timestamp + " ] " + message + "\n"));

}

private synchronized void broadcastMessage(String message, ClientHandler sender)

{ log("Broadcasting: " + message);

    for (ClientHandler client : clients) {

        // Send to all clients except the sender

```

```

        if (client != sender) {

            client.sendMessage(message);

        } }

private synchronized void removeClient(ClientHandler client) {

    clients.remove(client);

    log("Client disconnected: " + client.socket.getInetAddress().getHostAddress());

}

// --- Inner class to handle each client connection

---class ClientHandler implements Runnable {

    private Socket socket;

    private PrintWriter out;

    private BufferedReader in;

    private String clientName;

    public ClientHandler(Socket socket) {

        this.socket = socket;

    }

    @Override

    public void run() {

        try {

            out = new PrintWriter(socket.getOutputStream(), true);

            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

            // First message from client is their name

```

```

        clientName = in.readLine();

        if(clientName == null || clientName.trim().isEmpty()){

            clientName = "Anonymous@" + socket.getInetAddress().getHostAddress();

        }

        broadcastMessage(clientName + " has joined the chat.", this);

        log(clientName + " has set their name.");

        String message;

        while ((message = in.readLine()) != null)

            { broadcastMessage(clientName + ": " + message, this);

            }

    } catch (IOException e) {

        // Client disconnected or an error occurred

    } finally {

        // Cleanup

        removeClient(this);

        broadcastMessage(clientName + " has left the chat.", null); // Broadcast to all
remaining clients

        try {

            socket.close();

        } catch (IOException e)

            { e.printStackTrace();

            }

        }

    }
}

```

```
public void sendMessage(String message)

    { out.println(message);

    }

}

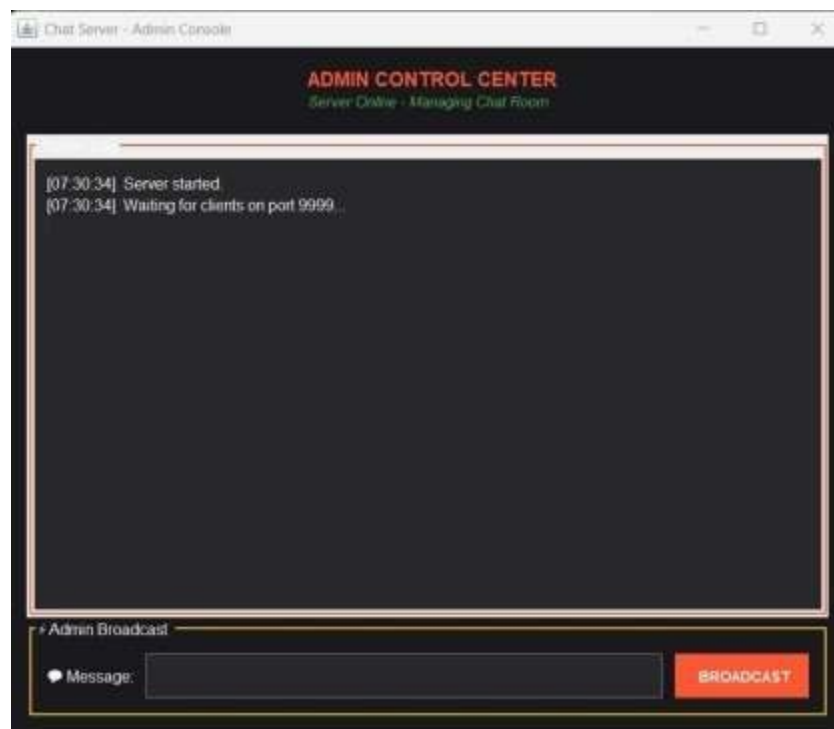
public static void main(String[] args)

    { new Server();

    }

}
```

7. PROJECT OUTPUT



Chat Setup

Enter Server IP Address:

127.0.0.1

OK Cancel

Chat Setup

Enter your name:

Guest

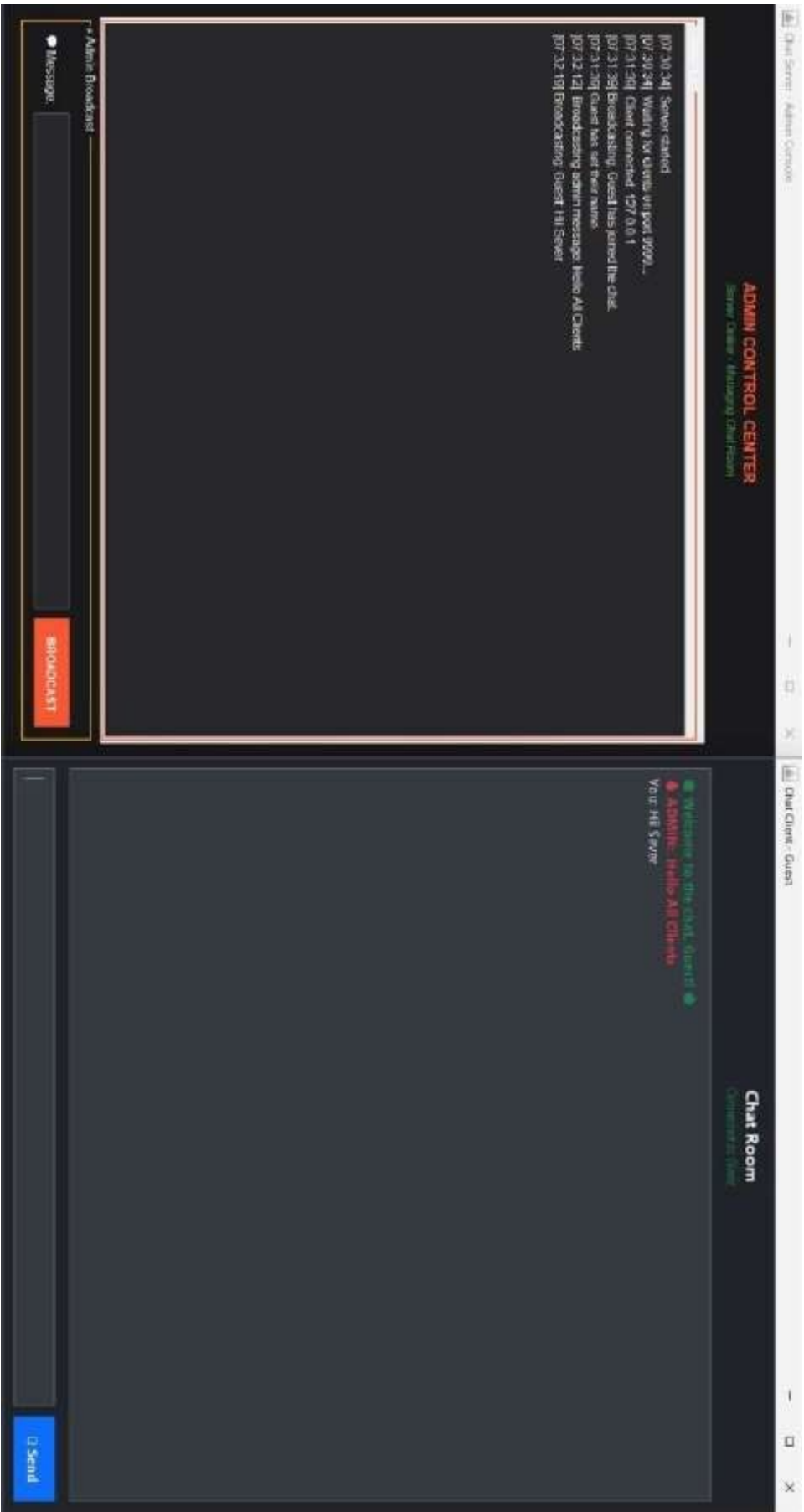
OK Cancel

Chat Client - Guest

Chat Room
Connected as Guest

★ Welcome to the chat, Guest! ★

Send



Application Description

The project consists of two runnable applications: the server and the client.

1. Server Application When launched, the server application opens a window titled "Chat Server". This window contains a large text area that serves as a log. It displays status messages in real-time, such as:

- "Server started."
- "Waiting for clients on port 9999..."
- "Client connected: 127.0.0.1"
- "[ClientName] has set their name."
- "Broadcasting: [ClientName]: Hello everyone!"
- "Client disconnected: 127.0.0.1"

2. Client Application When a user runs the client application:

- An initial dialog box appears, prompting for the "Server IP Address".
- A second dialog box asks the user to "Enter your name".
- After providing the details, the main chat window opens, titled "Chat Client - [YourName]".
- This window has a large central area to display the chat history.
- At the bottom, there is a text field where the user can type their message and a "Send" button to send it. Pressing Enter also sends the message.
- Messages sent by the user appear as "You: [Your Message]".
- Messages received from others appear as "[Sender's Name]: [Their Message]".
- System messages like "[Another User] has joined the chat" are also displayed.

8. ADVANTAGES AND DISADVANTAGES

Advantages

- **Platform Independence:** Being a Java application, both the client and server can run on any operating system with a Java Runtime Environment (JRE) installed.
- **Real-time Communication:** The application facilitates instant message delivery between users.
- **Scalability:** The multithreaded server architecture allows it to handle many client connections simultaneously, making it scalable.
- **Centralized Control:** The server acts as a central authority, simplifying the logic for message distribution and user management.
- **Lightweight:** The application uses core Java libraries and has minimal overhead, requiring no external dependencies.

Disadvantages

- **No Message Persistence:** Chat history is not saved. If the server restarts, all conversation history is lost.
- **Basic Security:** Communication is sent as plain text over the network, with no encryption. This makes it vulnerable to eavesdropping.
- **Basic User Interface:** Java Swing provides a functional but visually dated UI compared to modern web or mobile frameworks.
- **No Advanced Features:** The application lacks features common in modern chat apps, such as private messaging, user profiles, or file sharing.
- **Requires Running Server:** The application is entirely dependent on a central server that must be running and accessible to all clients.

9. CHALLENGES FACED

1. Handling Multiple Clients Concurrently

- **Challenge:** A server must be able to listen for new connections while simultaneously handling communication with already connected clients. A simple, single-threaded approach would block the entire application while waiting for a single client's message.
- **Solution:** A multithreaded architecture was implemented. The server's main thread is dedicated to accepting new Socket connections. Each accepted connection is then handed off to a new `ClientHandler` thread, which manages all I/O for that specific client independently.

2. Ensuring UI Responsiveness on the Client Side

- **Challenge:** The client application needs to listen for incoming messages from the server continuously. If this network listening operation is performed on the main GUI thread (the Event Dispatch Thread or EDT), the user interface would freeze and become unresponsive.
- **Solution:** A separate background thread (`ServerListener`) was created on the client side. This thread's sole responsibility is to block and wait for messages from the server's input stream. When a message arrives, it uses `SwingUtilities.invokeLater()` to queue the UI update (appending the message to the text area). This ensures that the GUI is modified safely on the EDT without blocking it.

3. Synchronizing Access to Shared Resources

- **Challenge:** The server maintains a shared `List` of all connected clients. Multiple `ClientHandler` threads need to access this list: to add new clients, to remove disconnected clients, and to iterate over it when broadcasting a message. Uncontrolled concurrent access could lead to `ConcurrentModificationException` or other race conditions.
- **Solution:** The methods that modify or iterate over the shared clients list (`broadcastMessage` and `removeClient`) were declared as synchronized. This keyword

acts as a mutex lock, ensuring that only one thread can execute these methods at a time, thus guaranteeing thread-safe access to the shared list.

10. CONCLUSION

The Client-Server Chat Application project was successfully designed and implemented, achieving all the primary objectives. It serves as a strong practical demonstration of core Java concepts, including socket programming for networking, multithreading for concurrency, and Swing for GUI development.

Through this project, we have gained valuable hands-on experience in building a distributed application from the ground up. We learned how to manage multiple concurrent connections on a server, how to maintain a responsive user interface during network operations, and the importance of thread synchronization when dealing with shared resources.

While the application is basic in its features, it provides a solid and extensible foundation. It successfully showcases the power and versatility of Java for network programming and can be further developed to include advanced features like encryption, private messaging, and persistent storage.

11. REFERENCE

- Head First Java, 2nd Edition by Kathy Sierra & Bert Bates.
- Oracle Java Documentation: <https://docs.oracle.com/en/java/>
- GeeksforGeeks - "Multi-threaded chat application in Java": <https://www.geeksforgeeks.org/multi-threaded-chat-application-in-java/>